

25 AI INTERVIEW QUESTIONS WITH ANSWERS



MEDIUM TO HARD LEVEL QUESTIONS
FOR REAL-WORLD AI ROLES



PRACTICAL & RELEVANT

Real interview questions asked
in top AI & ML roles.



CONCEPTS THAT MATTER

Focus on deep understanding,
not just definitions.



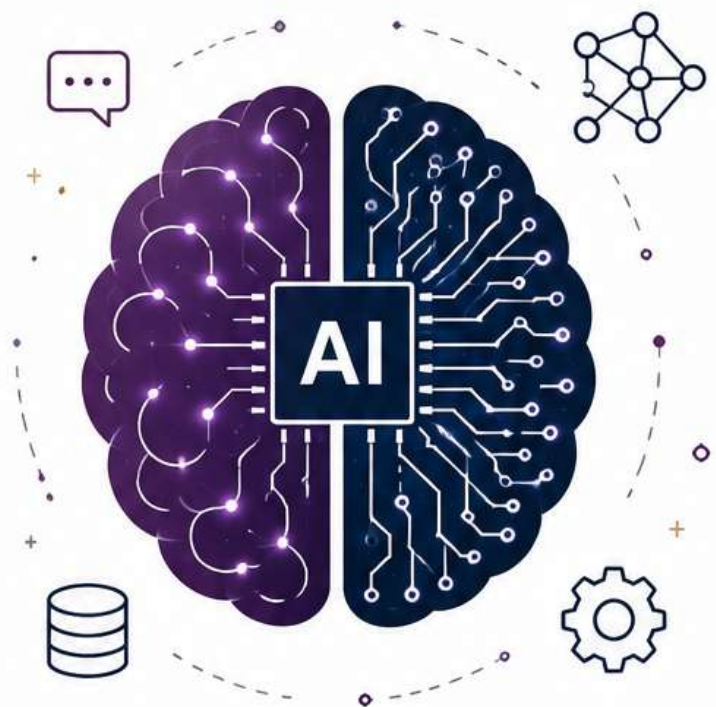
COVERS KEY AREAS

ML, Deep Learning, LLMs,
RAG, Agents, Evaluation,
Deployment & more.



LEVEL UP YOUR PREP

Sharpen your thinking and
stand out in your next
interview.



PREPARE SMART. THINK DEEP. BUILD THE FUTURE.

Let's get started!



TOPIC:

ML MODEL BEHAVIOR & GENERALIZATION (Q1–Q2)

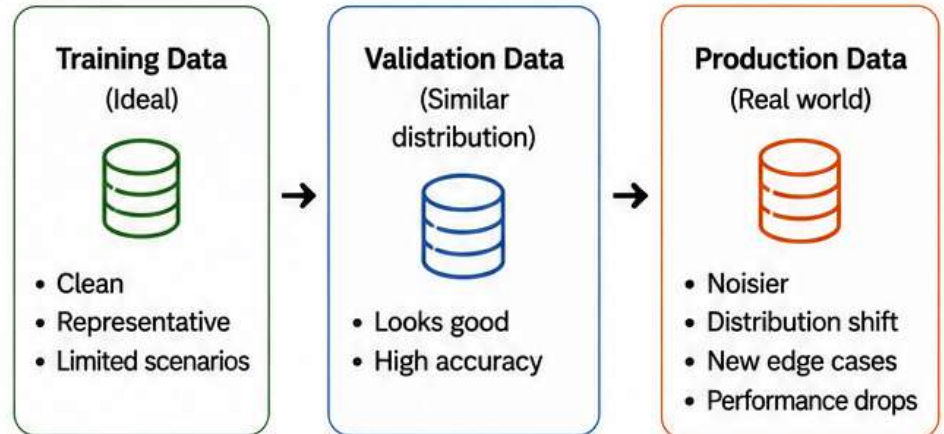
Q1

Why can a model have high validation accuracy but still perform poorly in production?

ANSWER:

High validation accuracy doesn't guarantee good real-world performance because:

- 1 Distribution shift (train/validation $\neq$ production data).
- 2 Data leakage in validation.
- 3 Overfitting to validation set or unrepresentative split.
- 4 Real-world data is noisier, has edge cases and out-of-distribution samples.



KEY TAKEAWAY: A model performs well on validation when the data is similar to training, but production introduces new distributions, noise and edge cases that the model hasn't seen.

Q2

How would you choose the right evaluation metric for a machine learning problem?

ANSWER:

Choose the metric based on the problem type, business objective and cost of errors.

- 1 Understand the business goal.
- 2 Consider class imbalance (if any).
- 3 Evaluate the cost of false positives vs false negatives.
- 4 Use multiple metrics when needed.

Problem Type	Common Metrics	When to Use
 Classification (Balanced)	Accuracy, F1-score	When classes are roughly balanced.
 Classification (Imbalanced)	Precision, Recall, F1-score, PR-AUC	When one class is rare.
 Regression	MAE, RMSE, R^2	When you need to measure prediction error.
 Ranking / Retrieval	Precision@K, Recall@K, NDCG	When order of results matters.



KEY TAKEAWAY: There is no single best metric. The right metric depends on the problem, data and business impact of different types of errors.



DATA



AI



TECH



ANALYTICS



SAVE FOR LATER



TOPIC:
Training, Optimization
& Regularization (Q3–Q4)

Q3

How would you diagnose training instability in a deep neural network?

ANSWER:

Check for these common causes:

- 1 Learning rate too high causing divergence.
- 2 Poor weight initialization leading to exploding/vanishing activations.
- 3 Exploding gradients in deep or RNN models.
- 4 Inappropriate optimizer settings (e.g., momentum, betas).
- 5 Numerical issues (NaNs/Infs) in loss or activations.

How to diagnose:



Monitor Loss Curves

Look for oscillations, plateaus, or divergence.



Inspect Gradients

Check gradient norms. Exploding (very high) or vanishing (near zero).



Track Activations

Check mean and variance across layers for saturation or collapse.



Check for NaNs/Infs

Identify the layer/operation where they first appear.



Ablate & Tune

Reduce LR, adjust batch size, try gradient clipping, change optimizer.



KEY TAKEAWAY: Training instability usually comes from learning rate, initialization, gradients, or numerical issues. Systematically monitor, measure, and adjust to stabilize training.

Q4

What is the difference between L1 and L2 regularization, and when would you use each?

ANSWER:

Both prevent overfitting but work differently.

- 1 L1 (Lasso) adds the sum of absolute weights.
- 2 L2 (Ridge) adds the sum of squared weights.
- 3 They impact the model and feature selection differently.

Aspect	L1 Regularization (Lasso)	L2 Regularization (Ridge)
 Penalty Term	$\lambda \sum w_i $	$\lambda \sum w_i^2$
 Effect on Weights	Drives some weights exactly to zero (sparse model).	Shrinks weights towards zero but never exactly zero.
 Feature Selection	Performs feature selection (useful with high-dimensional data).	Keeps all features but reduces their impact.
 When to Use	When you suspect many irrelevant features. Need interpretability.	When all features contribute small. When multicollinearity exists.
 Geometry	Diamond constraint (corners lead to zeros). 	Circular constraint (shrinks uniformly). 



KEY TAKEAWAY: Use L1 for sparsity and feature selection. Use L2 for generalization when all features carry signal.



DATA



AI



TECH



ANALYTICS



SAVE
FOR LATER



TOPIC:

Deep Learning (Q5–Q6)

Q5

What causes vanishing and exploding gradients, and how would you mitigate them?

ANSWER:

They occur when gradients become too small or too large while backpropagating through many layers.

Causes:

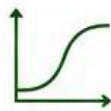
- 1 Repeated multiplication of small values (<1) → vanishing gradients.
- 2 Repeated multiplication of large values (>1) → exploding gradients.
- 3 Deep networks, poor initialization, or unsuitable activation functions.

How to mitigate:



Use Better Initialization

Use Xavier/He initialization to keep signal stable.



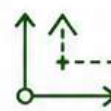
Choose Proper Activations

Use ReLU, Leaky ReLU, GELU instead of Sigmoid/tanh.



Normalize Activations

Use Batch Normalization or Layer Normalization.



Gradient Clipping

Clip gradients to a threshold to prevent exploding updates.



Architectural Choices

Use Residual Connections (ResNets) or LSTM/GRU for long sequences.



KEY TAKEAWAY: Keep the flow of gradients in a healthy range. Stabilize the signal, normalize, and clip when needed.

Q6

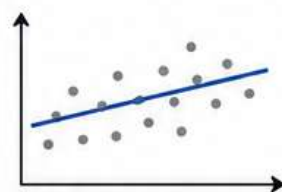
Explain the difference between underfitting, overfitting, and good fit with examples.

ANSWER:

These describe how well a model learns patterns in data.

- 1 **Underfitting:** Model is too simple to capture patterns.
- 2 **Overfitting:** Model learns noise and memorizes training data.
- 3 **Good fit:** Model captures true patterns and generalizes well.

Underfitting



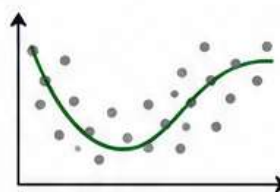
Model too simple (high bias)

- Fails to capture underlying pattern
- High training error and high test error

Example:

Linear model on highly non-linear data.

Good Fit



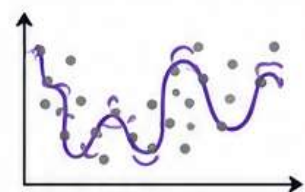
Right complexity (low bias, low variance)

- Captures true pattern
- Low training error and low test error

Example:

Polynomial regression with optimal degree.

Overfitting



Model too complex (high variance)

- Learns noise in training data
- Low training error but high test error

Example:

High-degree polynomial on small dataset.



KEY TAKEAWAY: Underfitting misses the signal. Overfitting chases the noise. Good models balance bias and variance for best generalization.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER



TOPIC:

CNNs, RNNs & Sequence Models (Q7–Q8)

Q7

What are the key differences between CNNs, RNNs, and Transformers?

ANSWER:

Each architecture is designed for different types of data and dependencies.



CNNs

Excel at capturing local spatial patterns in grid-like data (images, videos).



RNNs

Model sequential dependencies step-by-step (text, speech, time series).



Transformers

Use attention to model long-range dependencies in parallel (text, code, multimodal data).

	CNNs	RNNs	Transformers
Best For	Images, videos, spatial data	Sequences with order and timing	Long sequences, NLP, code, multimodal
How They Work	Convolutions extract local features	Hidden state updated step-by-step	Self-attention captures all positions at once
Dependency Handling	Local receptive fields	Sequential (dependent on previous state)	Global (all positions attend to each other)
Parallelization	High	Low (sequential)	High (fully parallel)
Strength	Efficient for spatial hierarchies	Good for temporal dynamics	Best for long-range context & scalability
Limitation	Not ideal for long sequences	Slow, suffers from vanishing gradients	Compute & memory intensive



KEY TAKEAWAY: CNNs focus on space, RNNs focus on time, Transformers focus on relationships. Choose based on data structure and dependency length.

Q8

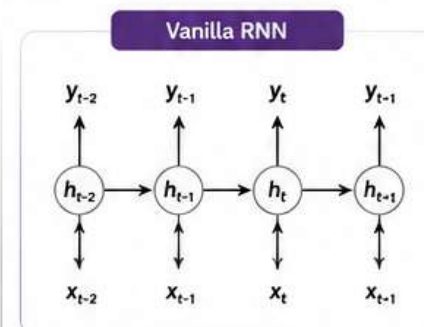
When would you use LSTM/GRU over a vanilla RNN?

ANSWER:

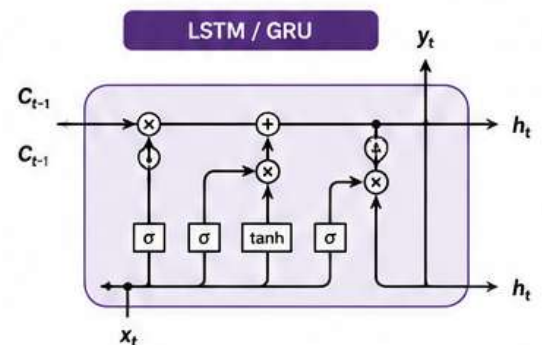
Use LSTM/GRU when sequences are long and vanilla RNN struggles to learn due to vanishing gradients.

Why They Help:

- 1 Gating mechanisms (in LSTM/GRU) control information flow.
- 2 Better at capturing long-term dependencies.
- 3 More stable training on long sequences.
- 4 Vanilla RNNs tend to forget important information over time.



Hard to pass information through many time steps.



Gates regulate what to keep, update, and forget.

Model	Key Idea	Pros	Cons	Use When
LSTM	Uses input, forget and output gates + cell state	Captures long-term dependencies very well	More parameters, slower training	Very long sequences, complex patterns
GRU	Simpler gates (reset and update gate)	Faster and fewer parameters	Slightly less expressive than LSTM	Long sequences, real-time applications



KEY TAKEAWAY: LSTM/GRU add gating to preserve important information over long sequences. Use them when vanilla RNN fails on long-term dependencies.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER



TOPIC:

Transformers & Attention (Q9–Q10)

Q9

What is the self-attention mechanism in Transformers, and why is it powerful?

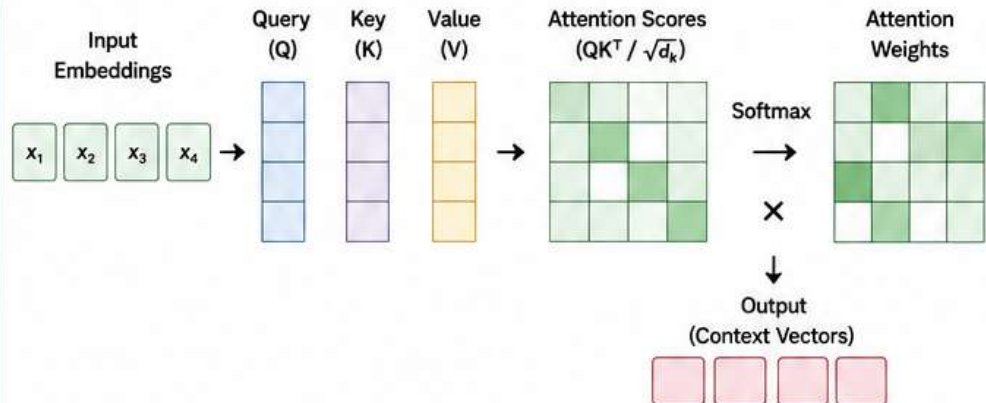
ANSWER:

Self-attention lets each position in a sequence attend to all other positions to compute a context-aware representation.

Why It's Powerful:

- 1 Captures long-range dependencies.
- 2 Fully parallelizable (unlike RNNs).
- 3 Learns which parts of the input are most relevant.
- 4 Scales better with more data and compute.

How Self-Attention Works



- Compute Q, K, V from embeddings
- Score similarity between Q and K
- Softmax to get attention weights
- Weight the values (V) to get output



KEY TAKEAWAY: Self-attention allows every token to gather information from all other tokens and focus on what matters most — regardless of distance.

Q10

What is the difference between multi-head attention and single-head attention?

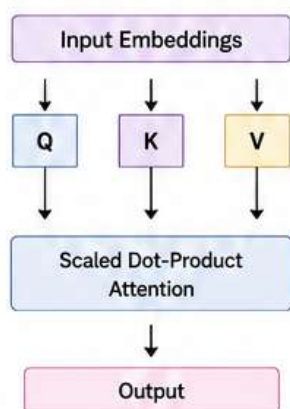
ANSWER:

Multi-head attention runs several self-attention heads in parallel, each learning different types of relationships.

Benefits of Multi-Head Attention:

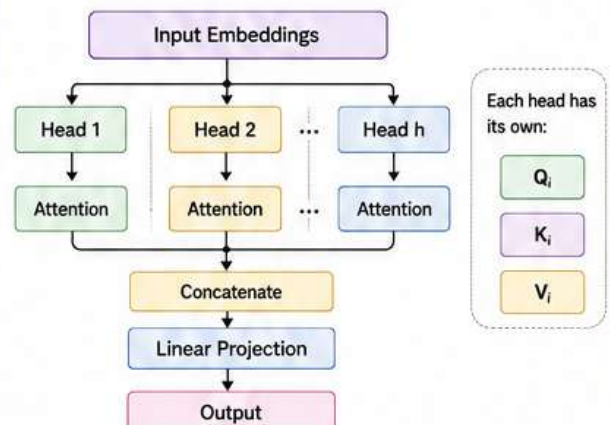
- 1 Captures diverse patterns at different positions.
- 2 Attends to information from different representation subspaces.
- 3 Improves model expressiveness and performance.
- 4 Helps the model focus on multiple aspects simultaneously.

Single-Head Attention



- ✗ Learns one type of relationship.
- ✗ Limited view of dependencies.

Multi-Head Attention



- ✓ Learns multiple types of relationships in parallel.
- ✓ Richer representations and better performance.



KEY TAKEAWAY: Multi-head attention lets the model look at the data from multiple perspectives at once, capturing more complex and complementary patterns.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER



Q11

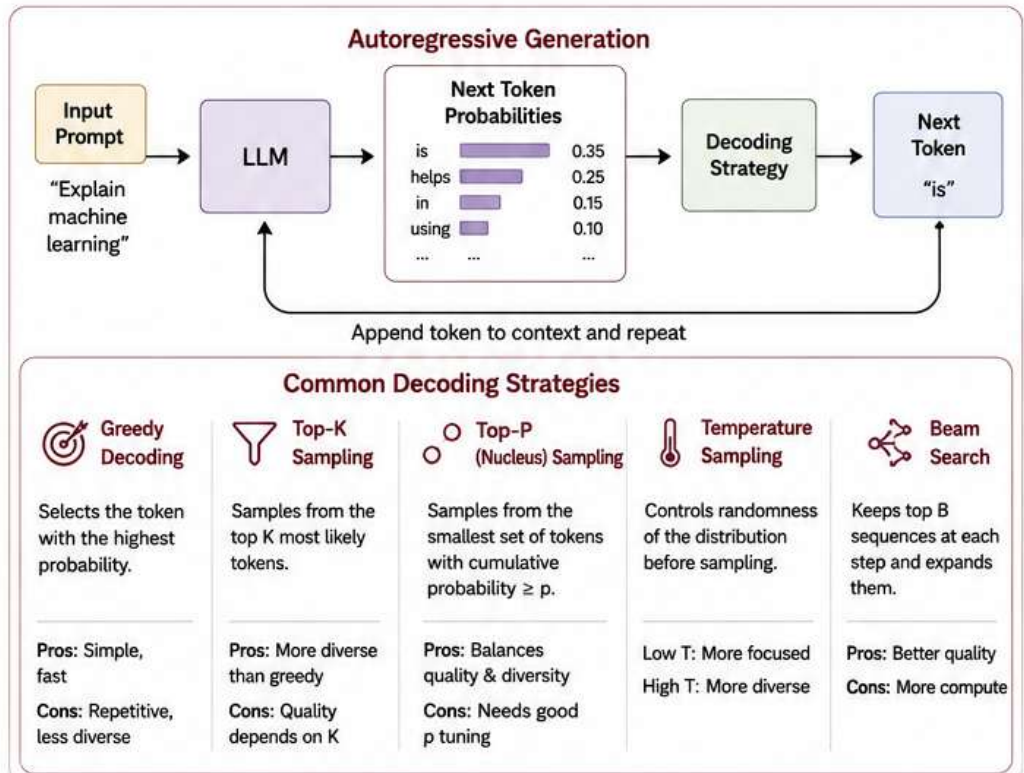
How do Large Language Models (LLMs) generate text, and what are the key decoding strategies?

ANSWER:

LLMs generate text one token at a time by predicting the next most likely token given the previous context.

Generation Process:

- 1 Input prompt is tokenized and passed to the model.
- 2 Model produces logits for the next token.
- 3 Logits are converted to probabilities.
- 4 A decoding strategy selects the next token.
- 5 The token is appended to the context and the process repeats.



KEY TAKEAWAY: LLMs generate text autoregressively. The decoding strategy you choose controls the trade-off between quality, diversity, and coherence.

Q12

What is the difference between fine-tuning, instruction tuning, and RLHF?

ANSWER:

These techniques improve LLM behavior at different stages and with different objectives.

	Fine-Tuning (SFT)	Instruction Tuning	RLHF (Reinforcement Learning from Human Feedback)
	Teach the model domain-specific knowledge or behavior.	Improve ability to follow instructions and generalize across tasks.	Align model responses with human preferences and values.
	Labeled input-output pairs (demonstrations).	Instruction-response pairs across diverse tasks.	Human preference data (ranked responses).
	Supervised fine-tuning on task-specific data.	Supervised fine-tuning on instruction datasets.	Train a reward model → use RL (e.g., PPO) to optimize policy.
	Model learns patterns and format of the data.	Model becomes better at following user instructions.	Model generates helpful, harmless, and honest outputs.
Example	Fine-tuning on legal documents Q&A dataset.	Alpaca, FLAN, OpenAssistant datasets.	ChatGPT, Claude, Gemini alignment.



KEY TAKEAWAY: Fine-tuning teaches, instruction tuning generalizes, and RLHF aligns. Together, they build powerful and human-aligned LLMs.



DATA



AI



TECH



ANALYTICS



SAVE
FOR LATER



TOPIC:

RAG, Embeddings & Vector Search (Q13–Q14)

Q13

What is Retrieval-Augmented Generation (RAG), and how does it work?

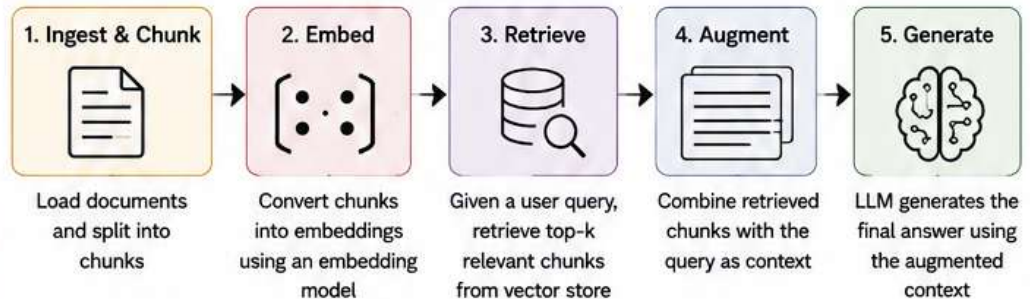
ANSWER:

RAG combines retrieval of relevant external information with LLM generation to produce more accurate, up-to-date, and grounded answers.

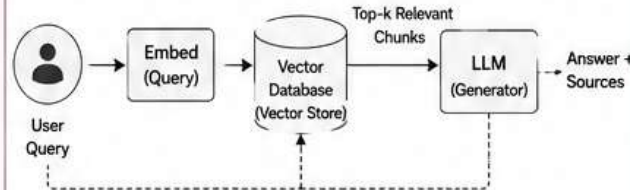
Why RAG?

- 1 Reduces hallucinations
- 2 Uses private/enterprise data
- 3 Keeps knowledge up-to-date
- 4 Provides source transparency

How RAG Works



RAG Architecture (High Level)



Key Choices in RAG

- ✓ Chunk size & overlap
- ✓ Embedding model
- ✓ Vector database (e.g., FAISS, Pinecone, Weaviate, Milvus)
- ✓ Retriever (top-k, MMR, hybrid)
- ✓ Re-ranker (optional)
- ✓ Prompt / context template



KEY TAKEAWAY: RAG grounds LLMs in external knowledge, improving accuracy and trust while keeping information current and traceable.

Q14

What is the difference between dense retrieval and sparse retrieval?

ANSWER:

Both retrieve relevant documents but use different representations and have different strengths.

When to Use What?



Use Sparse Retrieval for exact terms, codes, legal, or domain-specific queries.



Use Dense Retrieval for semantic search, conceptual queries, and natural language.



Hybrid Retrieval (Best of both): combines strengths and improves recall + precision.

Aspect	Sparse Retrieval (BM25 / TF-IDF)	Dense Retrieval (Embeddings)
Representation	Bag-of-words (term frequency based vectors)	Dense vector embeddings capturing semantic meaning
How it Works	Matches exact or partial keyword overlap	Matches semantic similarity in vector space
Strengths	• Great for exact term matches • Interpretable scores • Works well with rare keywords	• Captures synonyms & context • Better for natural language queries • Handles paraphrases well
Weaknesses	• Misses synonyms / meaning • Sensitive to wording • Lower recall on semantic queries	• May miss exact keywords • Embeddings can be costly • Harder to interpret
Best For	Legal docs, code search, FAQs, keywords, product IDs	Semantic search, Q&A, conversational search, recommendations
Examples	BM25, TF-IDF	SBERT, OpenAI text-embedding, Contriever

Hybrid Retrieval



KEY TAKEAWAY: Sparse is precise with keywords. Dense understands meaning. Hybrid retrieval gives the best balance of recall and relevance.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER



TOPIC:

Vector Databases & Similarity Search (Q15–Q16)

Q15

What are vector databases, and how do they enable similarity search?

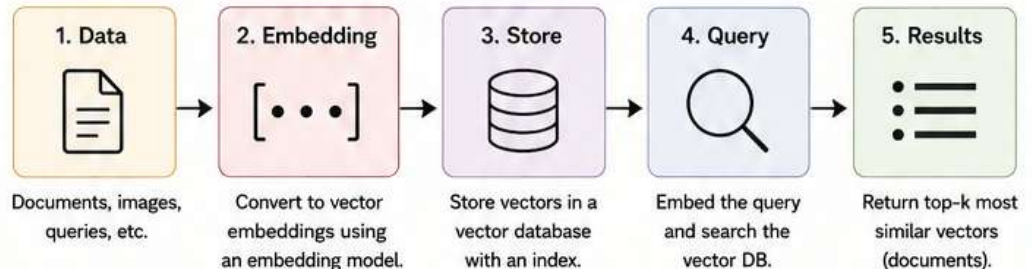
ANSWER:

Vector databases are built to store, index, and search high-dimensional embeddings efficiently using similarity metrics instead of exact matches.

Why They Matter:

- 1 Handle unstructured data (text, images, audio, etc.).
- 2 Power semantic search, recommendations, and RAG systems.
- 3 Scale to millions of vectors with sub-second search.

How Similarity Search Works



Common Similarity Metrics



Cosine Similarity — Measures angle between vectors.
Range: -1 (opposite) to 1 (same direction)



Dot Product — Measures magnitude & direction.
Higher = more similar.



Euclidean Distance — Measures straight-line distance.
Lower = more similar.

Popular Vector Databases



Pinecone — Managed, scalable, high performance



Weaviate — Open-source, hybrid search, filters



Milvus — Open-source, high throughput



Qdrant — Open-source, payload filtering



Chroma — Developer-friendly, easy to use



KEY TAKEAWAY: Vector databases let you find meaning, not just keywords, making search smarter and applications smarter.

Q16

What indexing techniques are used in vector databases for fast similarity search?

ANSWER:

Vector databases use specialized indexes to approximate nearest neighbor (ANN) search, balancing speed, memory, and accuracy.

Common Indexing Techniques:

- 1 **HNSW (Hierarchical Navigable Small World)**
Graph-based index. High recall with low latency.
- 2 **IVF (Inverted File Index)**
Clusters vectors using k-means and searches in relevant clusters.
- 3 **PQ (Product Quantization)**
Compresses vectors to reduce memory and speed up search.
- 4 **ScaNN / DiskANN**
Optimized for large-scale search with advanced pruning.

Index Type	How It Works	Strengths	Trade-offs	Best For
HNSW (Graph) 	Builds a multi-layer graph where each node connects to neighbors. Search navigates the graph.	✓ High recall ✓ Low latency ✓ Great for dynamic data	✗ Higher memory usage ✗ Slower build time	General purpose search, real-time applications
IVF (Cluster-based) 	Vectors are clustered. At query time, search only in the most relevant clusters.	✓ Scales to billions ✓ Faster search ✓ Lower memory	✗ Lower recall if clusters are not good ✗ Needs training	Large datasets, offline/online search
PQ (Compression) 	Vectors are compressed using quantization (sub-vectors).	✓ Reduces memory significantly ✓ Faster distance computations	✗ Some loss in accuracy ✗ Complex to tune	Memory-constrained environments, million+ vectors
ScaNN / DiskANN 	Uses optimized algorithms and pruning, sometimes leveraging SSD for extreme scale.	✓ Very fast ✓ Handles very large scale	✗ More complex infrastructure	Web-scale search, billions of vectors



KEY TAKEAWAY: Indexing is the key to fast similarity search—choose based on your scale, memory budget, and accuracy requirements.



DATA



AI



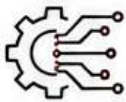
TECH



ANALYTICS



SAVE FOR LATER



TOPIC:

AI Agents & Agentic Systems (Q17–Q19)

Q17

What is an AI agent, and what are the core components of an agentic system?

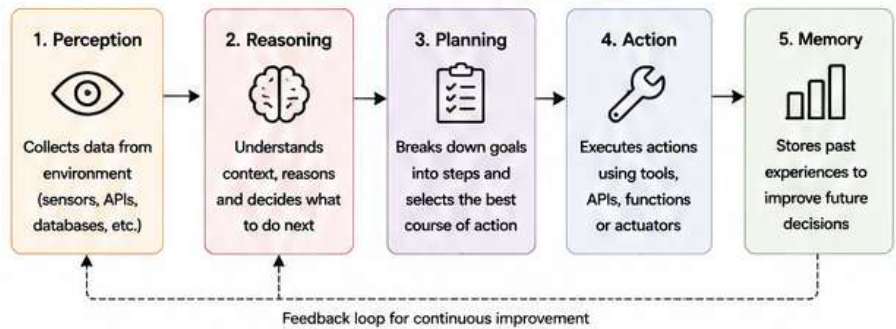
ANSWER:

An AI agent is an autonomous entity that perceives its environment, reasons about it, and takes actions to achieve goals.

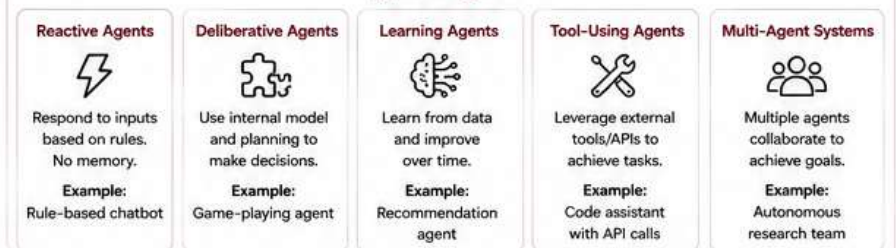
Key Characteristics:

- 1 **Autonomy** – operates with minimal human intervention.
- 2 **Goal-oriented** – works towards specific objectives.
- 3 **Perception** – understands environment via data/inputs.
- 4 **Reasoning** – plans and makes decisions.
- 5 **Action** – executes and learns from outcomes.

Core Components of an Agentic System



Types of Agents



Q18

What are the key challenges in and deploying AI agents in real-world applications?

ANSWER:

Building effective agents requires solving real-world challenges across reliability, safety, memory, tools, and scale.

Major Challenge Areas:

- 1 Reliability & Robustness
- 2 Safety & Alignment
- 3 Memory & Context Management
- 4 Tool Use & Integration
- 5 Scalability & Cost
- 6 Evaluation & Debugging
- 7 Security & Privacy
- 8 Human-in-the-Loop

Challenge	Why It's Hard	Impact	What Helps
Reliability & Robustness	Agents can fail on edge cases, ambiguous inputs, or out-of-distribution scenarios.	Incorrect actions, unstable behavior.	Better reasoning, guardrails, testing on diverse scenarios.
Safety & Alignment	Ensuring agents act in line with human values and intent is difficult.	Harmful or misaligned actions.	Alignment tuning, safety filters, clear objectives.
Memory & Context Management	Long-context handling and relevant memory retrieval is complex.	Forgets important info or gets confused.	Memory hierarchies, summarization, vector memory, context pruning.
Tool Use & Integration	Integrating tools, APIs, and systems can be unreliable.	Failed tool calls, wrong outputs.	Reliable tool schemas, retries, validation & fallbacks.
Scalability & Cost	Complex agents need more tokens, compute and infra.	High latency and operational cost.	Efficient models, caching, batching, distillation.
Evaluation & Debugging	Hard to define success metrics and trace multi-step reasoning.	Bugs hard to find, low trust in outputs.	Observability, tracing, eval benchmarks, simulation.
Security & Privacy	Agents can leak data, be prompt-injected, or misused.	Data breaches, security risks.	Access controls, sandboxing, input validation, prompt hardening.
Human-in-the-Loop	Fully autonomous agents may still need human oversight.	Lack of accountability, loss of control.	Human review, approvals, feedback loops.

Q19

How do multi-agent systems work, and what are their key design patterns?

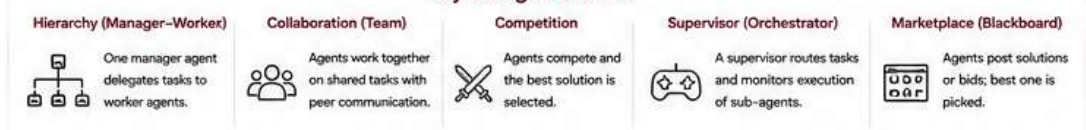
ANSWER:

Multi-agent systems involve multiple agents that collaborate, delegate, or compete to achieve goals more effectively than a single agent.

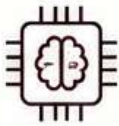
How Multi-Agent Systems Work



Key Design Patterns



KEY TAKEAWAY: AI agents perceive, reason, plan and act. Real-world deployment needs reliability, safety, memory, and scale. Multi-agent systems unlock greater capability through collaboration and smarter design patterns.



TOPIC:

Evaluation, Safety & Future of Generative AI (Q20–Q22)

Q20

How do you evaluate LLMs and generative AI systems?

ANSWER:

Evaluation looks at model quality, safety, reliability, and usefulness across different dimensions.

Key Evaluation Dimensions:

- 1 **Quality** – accuracy, relevance, fluency, coherence.
- 2 **Safety** – toxicity, bias, harmful content.
- 3 **Reliability** – factuality, consistency, hallucination rate.
- 4 **Robustness** – performance across edge cases & diverse inputs.
- 5 **User Experience** – helpfulness, clarity, satisfaction.

Evaluation Approaches & Metrics

Automatic Metrics	Human Evaluation	Task-Based Evaluation	Safety Evaluation	Online / A/B Testing
Use algorithms to score outputs.	Humans judge quality and safety.	Measure success on real tasks.	Check for risks and red flags.	Measure in real user environments.
• Perplexity (PPL) • BLEU / ROUGE • BERTScore • Exact Match • Faithfulness (e.g., LLM-Judge)	• Relevance • Helpfulness • Coherence • Factuality • Overall Rating	• QA Accuracy • Information Extraction F1 • Code Pass@k • Tool Usage Success	• Toxicity Scores • Bias / Fairness • Jailbreak Tests • Policy Compliance • Red Teaming	• User Feedback • Engagement • Conversion • Retention • Issue Rate



Good evaluation is multi-dimensional. No single metric captures everything—combining automated metrics, human judgment, and real-world feedback gives the truest picture.

Q21

What are the key safety risks in Generative AI, and how can we mitigate them?

ANSWER:

Generative AI can produce harmful, biased, or misleading outputs. Mitigation requires a combination of technical, process, and policy controls.

Key Safety Risks:

- Harmful Content** – violence, hate speech, self-harm, illegal instructions.
- Bias & Fairness** – stereotypes, unfair or discriminatory outputs.
- Misinformation** – hallucinations, false or misleading information.
- Privacy Leakage** – generating or exposing sensitive or personal data.
- Misuse & Jailbreaks** – prompt injection, jailbreaks, and policy bypass.

Mitigation Strategies

1. Data Governance	2. Model Alignment	3. Safety Filters	4. Red Teaming & Eval	5. Guardrails & Controls	6. Monitoring & Feedback
Curate high-quality, diverse, and policy-compliant data. Remove unsafe content.	Use techniques like RLHF, Constitutional AI, and DPO to align with human values.	Apply input & output moderation, toxicity filters, PII detection, and policy screening.	Continuously test with adversarial prompts and measure safety metrics.	Use prompt constraints, retrieval grounding, tool restrictions and escalation paths.	Monitor in production, collect feedback, and improve continuously.



Safety is not a one-time fix—it's an ongoing process. People, policy, and product must work together.

Q22

What are emerging trends and the future of Generative AI & LLMs?

ANSWER:

The future of Generative AI is multimodal, more capable, more efficient, and deeply integrated into real-world systems.

Key Trends Shaping the Future

1. Multimodal Models	2. Agentic AI & Automation	3. Long Context & Memory	4. Efficiency & Optimization	5. Personalization at Scale	6. Responsible & Trustworthy AI
Models that understand and generate text, images, audio, video, and beyond.	LLMs powering autonomous agents that plan, act, and collaborate.	Models handling much longer contexts with better memory and reasoning.	Smaller models, quantization, pruning, sparse models and edge deployment.	More personalized, context-aware experiences while protecting privacy.	Stronger safety, interpretability, governance and regulation.

Where It's Heading



From copilots to autonomous systems



From single models to multi-agent ecosystems



From cloud-only to edge + cloud hybrid



From capability races to responsible innovation



From tools to trusted partners



The future isn't just bigger models—it's better systems, safer deployment, and real impact.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER



TOPIC:

Advanced AI System Design (Q23–Q25)

Q23

How would you design a production-grade AI system that combines an LLM, RAG, tools, and human oversight?

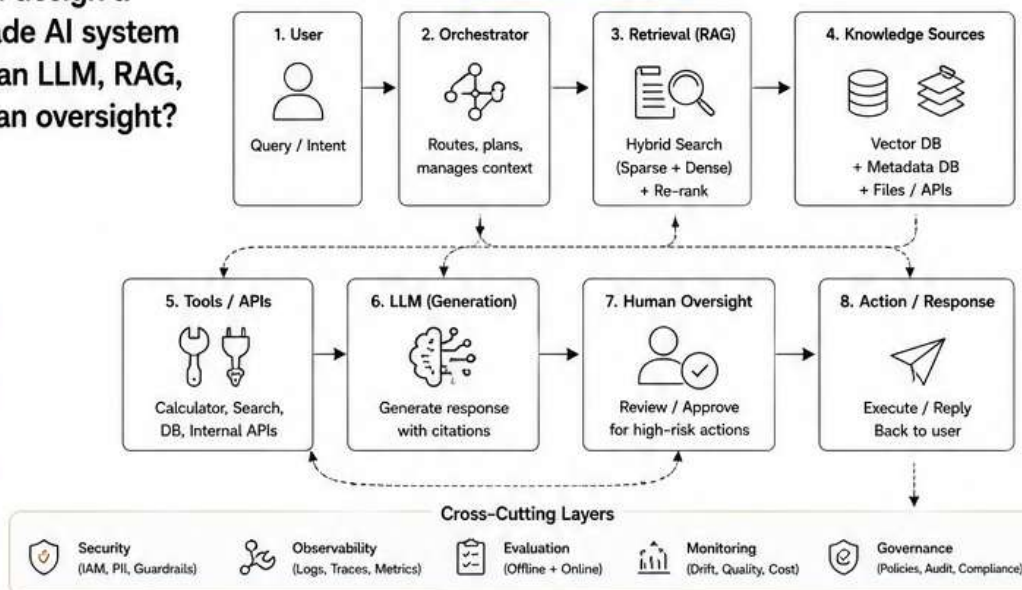
ANSWER:

Design for quality, reliability, safety and scalability from day one.

Key Design Principles

- Modular & loosely coupled
- Secure, observable & auditable
- Human-in-the-loop for critical actions
- Evaluated continuously
- Scalable, resilient & cost-aware

Reference Architecture



Q24

How would you reduce latency and cost in a large-scale LLM application without significantly hurting answer quality?

ANSWER:

Optimize across the model, retrieval, architecture, and operations layers.

Strategies to Reduce Latency & Cost

1. Smarter Retrieval	2. Prompt Optimization	3. Model Optimization	4. Caching	5. Architecture & Infra	6. Cost-Aware Routing
- Use hybrid search with re-ranking - Retrieve top-k only when needed - Compress context (summarization, extraction)	- Short, clear, structured prompts - Remove unnecessary instructions/examples - Use system prompts wisely	- Use smaller/cheaper models where possible - Distill from larger models - Quantization (8-bit/4-bit) - Choose right model for the task	- Cache frequent queries & embeddings - Cache tool/API responses - Use response templates	- Async & streaming responses - Batch requests intelligently - Use regional infra close to users	- Route simple queries to cheaper models - Escalate complex queries to stronger models



Golden Rule: Spend more compute only where it moves the needle on quality.

Q25

How would you design an evaluation and monitoring framework for an AI system after it goes into production?

ANSWER:

Continuously measure what matters, detect issues early, and close the loop with feedback.

Evaluation Framework

1. Offline Evaluation (Pre-Deployment)	2. Online Evaluation (Post-Deployment)	3. Continuous Feedback Loop
- Test datasets (golden set) - Metrics: Relevance, Factuality, Faithfulness, Coherence - Task metrics: Accuracy, F1, EM, etc. - Red teaming & safety tests	- A/B Testing - Interleaving / Shadow mode - Human ratings - Implicit signals: Clicks, Engagement, Conversions	- Capture user feedback - Log failure cases - Analyze & label - Improve data, prompts, models, and retrieval

Monitoring Framework

1. System Health	2. Model Performance	3. Data & Drift	4. Safety & Compliance	5. Cost & Efficiency
- Uptime, Latency - Error rates - Throughput	- Quality scores - Hallucination rate - Answer relevance	- Data freshness - Data drift - Concept drift	- Toxicity rate - PII leakage - Policy violations	- Cost per query - Token usage - Cache hit rate

Key Metrics to Track

Quality • Safety • Reliability • Latency • Cost • User Satisfaction • Business Impact



KEY TAKEAWAY:

Great AI systems are not just about smart models—they are about smart design, continuous evaluation, and responsible deployment at scale.



DATA



AI



TECH



ANALYTICS

SAVE
FOR LATER